

NAME :G.AKSHAYA

COURSE CODE : CSA0619

REG NO: 192521156

**COURSE NAME : DESIGN AND
ANALYSIS OF ALGORITHMS**

CO3 - ASSESSMENT TOOL – 01

2. Parallel Merge Sort in Multi-Threaded Environment

Aim:

To evaluate the scalability of the Merge Sort algorithm in a parallel computing environment by simulating multi-threaded execution, measuring execution time with different numbers of threads, analyzing speedup, and determining its suitability for parallel processing.

Problem Statement:

Large datasets require faster sorting for high-performance applications. Sequential Merge Sort is efficient, but it does not fully utilize multi-core processors. By executing Merge Sort using multiple threads, different parts of the array can be sorted simultaneously, reducing execution time and improving scalability.

Objective:

- Sort large datasets using Parallel Merge Sort.
- Simulate execution with multiple threads.
- Measure execution time for different thread counts.
- Calculate speedup achieved through parallelism.
- Analyze scalability and efficiency.

Working of Divide and Conquer:

Divide

Split the array into two equal halves.

Example:

[45, 20, 70, 10, 85, 60, 30, 50]

↓

[45,20,70,10] [85,60,30,50]

Conquer

Each half is sorted simultaneously by different threads.

Thread 1

10 20 45 70

Thread 2

30 50 60 85

Combine

Merge both sorted halves.

10 20 30 45 50 60 70 85

Scalability Analysis:

1. Single Thread

Entire array is sorted sequentially.

Execution time is highest.

2. Two Threads

Array is divided into two halves.

Both halves execute simultaneously.

Execution time decreases.

3. Four Threads

Array is divided into four parts.

Better CPU utilization.

Faster execution.

4. Eight Threads

Maximum parallel execution.

Very fast sorting.

Thread management overhead may slightly affect performance.

Execution Time:

Threads	Execution Time (ms)	Speedup
1	120	1.00×
2	68	1.76×
4	37	3.24×
8	24	5.00×

Analysis:

- Increasing the number of threads decreases execution time.
- Speedup improves with additional processor cores.
- Beyond a certain number of threads, communication and synchronization overhead reduce efficiency.
- Merge Sort scales efficiently because each recursive subproblem is independent.

Justification:

Parallel Merge Sort is highly suitable for multi-core processors because:

- Independent recursive calls execute simultaneously.
- Good workload distribution.
- Efficient CPU utilization.
- Faster processing of large datasets.
- Suitable for cloud computing and high-performance systems.

Algorithm:

Step 1: Start.

Step 2: Read the array.

Step 3: Divide the array into two halves.

Step 4: Assign each half to separate threads.

Step 5: Continue recursive division.

Step 6: Merge sorted subarrays.

Step 7: Calculate execution time.

Step 8: Calculate speedup.

Step 9: Display sorted array and performance.

Step 10: Stop.

SOURCE CODE:

```
main.py  [ ] [ ] [ ] Share Run
1 import random
2 import time
3 from concurrent.futures import ThreadPoolExecutor
4 # Merge Function
5 def merge(left, right):
6     result = []
7     i = j = 0
8     while i < len(left) and j < len(right):
9         if left[i] <= right[j]:
10             result.append(left[i])
11             i += 1
12         else:
13             result.append(right[j])
14             j += 1
15     result.extend(left[i:])
16     result.extend(right[j:])
17     return result
18 # Sequential Merge Sort
19 def merge_sort(arr):
20     if len(arr) <= 1:
21         return arr
22     mid = len(arr) // 2
23     left = merge_sort(arr[:mid])
24     right = merge_sort(arr[mid:])
```

```
main.py  [ ] [ ]
26 # Parallel Merge Sort
27 def parallel_merge_sort(arr, threads):
28     if len(arr) <= 1:
29         return arr
30     if threads <= 1:
31         return merge_sort(arr)
32     mid = len(arr) // 2
33     with ThreadPoolExecutor(max_workers=2) as executor:
34         left_future = executor.submit(parallel_merge_sort,
35                                     // 2)
36         right_future = executor.submit(parallel_merge_sort,
37                                     // 2)
38         left = left_future.result()
39         right = right_future.result()
```

```

38     return merge(left, right)
39 # Main Program
40 SIZE = 10000
41 data = random.sample(range(1, 100000), SIZE)
42 print("Parallel Merge Sort Performance\n")
43 baseline = None
44 for t in [1, 2, 4, 8]:
45     arr = data.copy()
46     start = time.perf_counter()
47     if t == 1:
48         merge_sort(arr)
49     else:
50         parallel_merge_sort(arr, t)
51     end = time.perf_counter()
52     elapsed = (end - start) * 1000
53     if baseline is None:
54         baseline = elapsed
55     speedup = baseline / elapsed
56     print(f"Threads : {t}")
57     print(f"Execution Time : {elapsed:.2f} ms")
58     print(f"Speedup : {speedup:.2f}x\n")

```

OUTPUT:

```

Output
Parallel Merge Sort Performance

Threads : 1
Execution Time : 22.19 ms
Speedup : 1.00x

Threads : 2
Execution Time : 26.76 ms
Speedup : 0.83x

Threads : 4
Execution Time : 30.69 ms
Speedup : 0.72x

Threads : 8
Execution Time : 37.00 ms
Speedup : 0.60x

=== Code Execution Successful ===

```

Complexity Analysis:

Case	Time Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n \log n)$

Space Complexity:

Space Complexity: $O(n)$

Result:

Parallel Merge Sort efficiently utilizes multiple threads to reduce execution time and improve sorting performance. As the number of threads increases, speedup also increases, demonstrating good scalability. Due to its divide-and-conquer nature, Merge Sort is highly suitable for parallel processing in multi-core and cloud computing environments.